

RestoMe

Application mobile React Native
Commande à table en temps réel

Dossier de remise — Projet React Native Ilhami Kandemir

Date limite : 28 mai 2026

1. Rappel du sujet

RestoMe est une application mobile React Native développée avec Expo. Elle permet aux clients d'un restaurant de scanner un QR code de table, rejoindre ou créer une session, consulter le menu, ajouter des plats avec des notes, envoyer une commande, suivre l'état des articles en temps réel et demander l'addition.

L'application contient aussi un mode cuisine caché, accessible uniquement via le QR code staff **KITCHEN_001**, où le personnel peut se connecter, voir les commandes en direct, changer le statut des articles, envoyer des messages aux clients, gérer le menu, les photos, les allergènes et la disponibilité des plats.

2. Fonctionnalités implémentées

Côté client

Fonctionnalité	Statut
Scan QR avec caméra	Réalisé avec expo-camera
Saisie manuelle du QR code	Réalisé
Détection QR cuisine caché	Réalisé
Création/reprise d'une session de table	Réalisé via Supabase
Menu par catégories	Réalisé : starter, main, dessert, drink
Détail d'un plat	Image, prix, allergènes, quantité, notes
Panier local	Réalisé avec React Context
Soumission de commande	Réalisé via Supabase
Historique local de commande	Réalisé avec SQLite
Suggestions basées sur l'historique	Réalisé
Allergènes sauvegardés localement	Réalisé
Mode clair / sombre	Réalisé avec SQLite
Suivi de commande en temps réel	Réalisé avec Supabase Realtime
Messages cuisine → client	Réalisé
Demande d'addition	Interface + alerte locale

Côté cuisine

Fonctionnalité	Statut
Connexion email / mot de passe	Réalisé avec Supabase Auth
Accès cuisine caché par QR code	Réalisé
Protection des routes cuisine	Réalisé dans le layout kitchen
Liste des tables actives	Réalisé
File de commandes en direct	Réalisé
Statistiques pending / preparing / ready	Réalisé

Fonctionnalité	Statut
Changement statut article	Réalisé
Marquer un article indisponible	Réalisé
Envoyer un message au client	Réalisé
Fermer une session	Réalisé
Ajouter/modifier un article menu	Réalisé
Upload image article	expo-image-picker + Supabase Storage
Ajouter un allergène personnalisé	Réalisé
Rechercher dans le menu cuisine	Réalisé
Activer/désactiver disponibilité	Réalisé en temps réel

3. Justification technique

Le projet utilise React Native avec Expo pour créer une application mobile rapidement testable avec Expo Go. J'ai choisi Expo Router plutôt que React Navigation parce que l'application a deux espaces clairement distincts (client et cuisine) avec des layouts indépendants. Expo Router permet d'exprimer cette séparation directement dans la structure de fichiers (app/ vs app/(kitchen)/) sans configuration manuelle d'un Navigator. Le grouping (kitchen) isole les routes cuisine sans les exposer dans l'URL client, ce qui correspond exactement à l'exigence de masquer le mode cuisine.

Le projet utilise TypeScript, ce qui permet de définir clairement les types métier : `MenuItem`, `Order`, `OrderItem`, `Session`, `Allergen`, etc. Cela renforce la robustesse du code et facilite la maintenance.

Base de données distante

Le projet utilise Supabase avec `@supabase/supabase-js`. J'ai choisi Supabase plutôt qu'une API REST custom ou Axios parce que le projet nécessite trois fonctionnalités simultanément : une base de données relationnelle, des mises à jour temps réel (suivi de commande client, file cuisine), et une authentification sécurisée. Supabase fournit ces trois éléments dans un seul client, avec Realtime intégré.

Authentification

L'authentification cuisine utilise Supabase Auth avec email/mot de passe. La session auth est persistée avec Expo SecureStore pour garantir un stockage sécurisé des tokens sur l'appareil.

Pour s'authentifier en tant que cuisine, il faut soit scanner le code QR `kitchen.svg` qui se trouve sous `src/qr-codes`, soit taper `KITCHEN_001` manuellement.

Stockage local

J'ai choisi SQLite plutôt qu'AsyncStorage parce que les données locales incluent un historique de commandes utilisé pour générer des suggestions personnalisées. AsyncStorage est un stockage clé-valeur qui nécessiterait de charger et désérialiser l'intégralité de l'historique pour le filtrer. SQLite permet d'écrire une requête `SELECT` directement sur la table `order_history`, ce qui est plus efficace et prévisible à mesure que l'historique grandit. Le menu et les commandes restent dans Supabase pour assurer la cohérence des données entre clients et cuisine.

État global

L'état global utilise React Context, organisé par responsabilité :

- **AuthContext** — état de connexion cuisine
- **SessionContext** — table/session client
- **CartContext** — panier local
- **ThemeContext** — thème clair/sombre
- **UserSettingsContext** — allergènes sauvegardés

J'ai choisi React Context plutôt que Redux parce que l'état global du projet est limité à cinq domaines indépendants (auth, session, panier, thème, allergènes), chacun avec une surface d'API réduite. Redux apporte une complexité justifiée pour des états larges avec beaucoup d'actions

croisées, ce qui n'est pas le cas ici. Context permet de couvrir ce besoin avec le code standard React, sans dépendance supplémentaire.

Formulaires

Les formulaires cuisine (login, création/édition d'article menu) auraient pu être implémentés avec `useState` + validation manuelle comme dans le cours. J'ai choisi `React Hook Form` + `Zod` parce que le formulaire menu contient une dizaine de champs avec des règles de validation interdépendantes (ex. : prix positif, URL image optionnelle, allergènes multiples). Avec `useState`, chaque champ aurait nécessité un état séparé et une validation inline, rendant le composant difficile à maintenir. `React Hook Form` gère l'état des champs hors du cycle de rendu React, ce qui évite un re-rendu à chaque frappe. `Zod` définit les règles de validation comme un schéma TypeScript réutilisable.

Temps réel

Le temps réel utilise Supabase Realtime :

- `order_items` — mise à jour de la file cuisine
- `status_updates` — affichage des messages côté client
- `menu_items` — propagation des changements de disponibilité
- `session_joins` — channel broadcast pour notifier la cuisine quand une session est ouverte ou rejointe

4. Architecture

Structure principale

Dossier / Fichier	Rôle
app/	Routes Expo Router
app/(kitchen)/	Routes du mode cuisine
src/features/customer/	Écrans, composants et hooks client
src/features/kitchen/	Écrans, composants, hooks et utils cuisine
src/services/	Accès Supabase : menu, commandes, sessions, tables, auth
src/contexts/	États globaux React Context
src/lib/db.ts	Initialisation et accès SQLite
src/lib/supabase.ts	Client Supabase
src/types/index.ts	Types TypeScript partagés

Flux client

- Le client ouvre l'app sur l'écran QR.
- Il scanne un QR code de table ou saisit le code manuellement (les QR codes se trouvent sous `src/qr-codes`). Les codes valides sont : `TABLE_001`, `TABLE_002`, `TABLE_003`, `TABLE_004`, `TABLE_005`, `TABLE_006`.
- L'app cherche la table dans Supabase via `tables.qr_code`.
- Elle vérifie s'il existe une session ouverte ; si oui, le client peut la rejoindre, sinon une nouvelle session est créée.
- Le client consulte le menu, ajoute des articles au panier, puis valide.
- L'app crée ou retrouve la commande ouverte de la session.
- Les articles sont insérés dans `order_items` avec le statut `pending`.
- Le client suit l'état en direct sur l'écran *live order*.

Flux cuisine

- Le staff scanne le QR cuisine `KITCHEN_001`.
- L'app redirige vers `/login`.
- Le staff se connecte avec Supabase Auth.
- Le dashboard affiche les sessions ouvertes et commandes actives.
- Le staff change les statuts : `pending`, `preparing`, `ready`, `unavailable`.
- Le staff peut envoyer des messages au client via `status_updates`.
- Le staff peut fermer une session, ce qui ferme aussi la commande ouverte liée.

5. Auto-évaluation

Le tableau ci-dessous récapitule les fonctionnalités annoncées et leur état de réalisation effectif :

Fonctionnalité annoncée	Réalisée ?	Commentaire
Scan QR table	Oui	Caméra + saisie manuelle
Session par table	Oui	Supabase sessions
Menu par catégories	Oui	starter, main, dessert, drink
Détail plat avec allergènes	Oui	Image, prix, allergènes
Panier local	Oui	React Context
Notes par article	Oui	Stockées dans order_items.notes
Soumission commande	Oui	Insertion Supabase
Commandes immuables côté client	Oui	Pas d'édition après envoi
Suivi temps réel client	Oui	order_items + status_updates
Suggestions personnalisées	Oui	Basées sur SQLite order_history
Thème clair/sombre	Oui	Sauvegardé en SQLite
Préférences allergènes locales	Oui	SQLite user_allergens
Connexion cuisine	Oui	Supabase Auth
Mode cuisine caché	Oui	QR staff uniquement
Dashboard cuisine live	Oui	Supabase Realtime
Changement statut article	Oui	Update order_items.status
Messages cuisine-client	Oui	Table status_updates
Gestion du menu	Oui	Création, édition, disponibilité
Upload photo menu	Oui	Supabase Storage
Ajout allergène	Oui	Table allergens
Demande d'addition	Partiel	Interface/alerte locale, pas persistée en base

6. Configuration à documenter

Le projet nécessite un fichier `.env` à la racine avec les variables suivantes (le fichier `.env` sera dans le fichier zip) :

```
EXPO_PUBLIC_SUPABASE_URL= https://cyixryvxwdvttpkhszmw.supabase.co
EXPO_PUBLIC_SUPABASE_ANON_KEY=sb_publishable_2b5HD-Zjw_6SJwfqgwD4tQ_6Xog8THZ
```

Commandes utiles

Commande	Description
<code>npm install</code>	Installe les dépendances du projet.
<code>npm run start</code>	Lance le serveur de développement Expo.
<code>npm run start:tunnel</code>	Lance Expo en mode tunnel (test sur appareil physique avec Expo Go, notamment lorsque le réseau local n'est pas accessible).
<code>npm run ts:check</code>	Vérifie les types TypeScript de l'ensemble du projet.
<code>npm test</code>	Exécute la suite de tests.

Notes importantes sur le lancement

Remarque : il arrive que la commande `npm run start:tunnel` ne fonctionne pas correctement à cause des configurations ngrok ; aucune solution stable n'a pu être trouvée pour ce problème.

Solution recommandée : Si vous avez l'erreur body ngrok, vous devez supprimer toutes vos terminal car un tunnel est déjà ouvert quelque part (si le problème persiste, reset le pc) Sinon essayez de lancer via Android Studio directement ou par câble usb

7. Informations évaluateur

Cette section regroupe toutes les informations nécessaires à l'évaluateur pour reproduire la démonstration de l'application lors de la soutenance.

7.1 Identifiants de démonstration cuisine

```
Email : staff@resto.me  
Mot de passe : test1234
```

7.2 Flux de démonstration recommandé

Pour présenter l'application de bout en bout pendant la soutenance, le déroulé conseillé est le suivant :

1. Lancer l'application avec `npm run start`.
2. Scanner ou taper `TABLE_001`.
3. Aller dans ses paramètres pour configurer ses allergènes pour filtrer automatiquement les plats ou changer de thème.
4. Ajouter un article du menu au panier.
5. Envoyer la commande.
6. Cliquer sur « Mes commandes » pour regarder le statut de la commande et voir les changements en directe
7. Sur un deuxième téléphone (pour mieux voir le realtime), scanner ou taper `KITCHEN_001`.
8. Se connecter avec le compte cuisine de démonstration.
9. Ouvrir la file de commandes en direct.
10. Interagir avec les commandes d'une table
11. Changer le statut de l'article : `pending` → `preparing` → `ready`.
12. Aller sur « Mon menu »
13. Interagir avec les différents composants (créer un allergène, créer un produit, uploader une image, activer ou désactiver un produit)

7.3 Limitations connues

- L'application nécessite une connexion internet active.
- Pas de file de commandes hors-ligne.
- Le client ne peut pas modifier ou annuler une commande déjà envoyée.
- La demande d'addition reste un retour d'interface local, non persisté dans Supabase.

7.4 Notes de lancement

- Utiliser Node.js **20.19.4** ou une version plus récente.
- Exécuter `npm install`.
- Exécuter `npm run start`.
- Si le téléphone / Expo Go n'arrive pas à se connecter, essayer `npm run start:tunnel`.
- Si le tunnel échoue (problèmes de configuration ngrok connus) fermer toutes les terminaux (qui ont des tunnels actifs) ou restart son pc. Si rien ne fonctionne, utiliser un émulateur Android Studio et appuyer sur la touche **a** dans le terminal Expo pour ouvrir l'application sur l'émulateur.

8 . QR codes de test

QR codes client (tables) :

- TABLE_001



- TABLE_002



- TABLE_003



- TABLE_004



- TABLE_005



- `TABLE_006`



QR code cuisine / staff :

- `KITCHEN_001`



Les fichiers SVG correspondants se trouvent aussi dans `src/qr-codes`. Les codes peuvent aussi être saisis manuellement depuis l'écran d'accueil de l'application.